Why Evaluate Machine Learning Models?

Improved decision-making: Evaluation shows how **well** a model works, guiding choices about **deployment** and **improvement.**

Overfitting Prevention: Evaluation can warn against models that might **memorize training** data but do **poorly** on **new data**.

Fine-tuning: Assessment enables us to **tweak** the model improving its effectiveness.

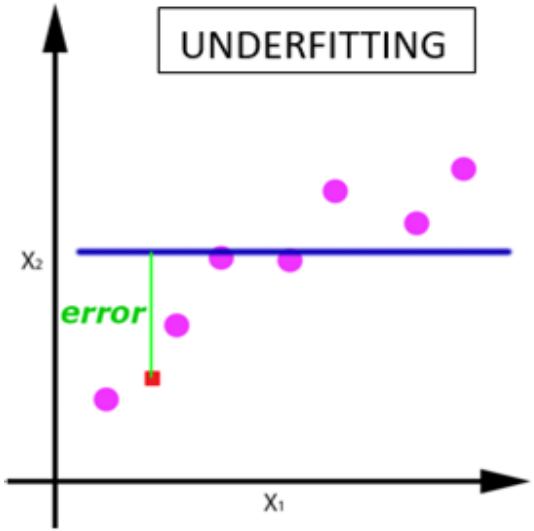
Fair Comparison: It enables us to objectively **compare different models**.

What is Underfitting?

Too Simple: An underfit model **fails** to **capture** the underlying **pattern of the data**.

Poor Performance: This leads to errors on both the **training set** and unseen data.

Example: Trying to fit a straight line to data that has a clear curve.

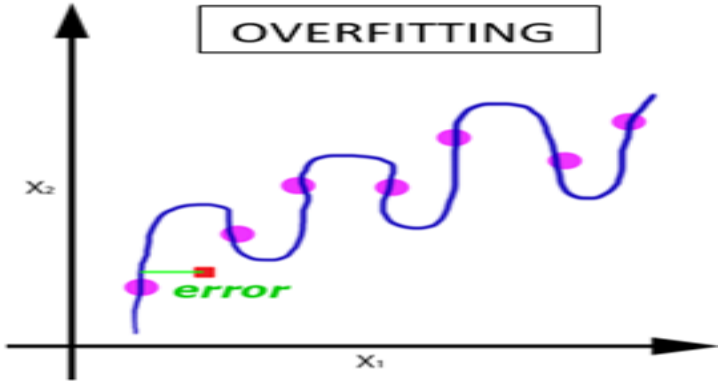


What is Overfitting?

Too Complex: An overfit model **learns** the training data **too well**.

Memorization, not Generalization: It becomes a **lookup table** rather than a **pattern detector**.

Poor performance on new data: The model becomes overly specific to the training data and makes **bad predictions** on **unseen examples**.



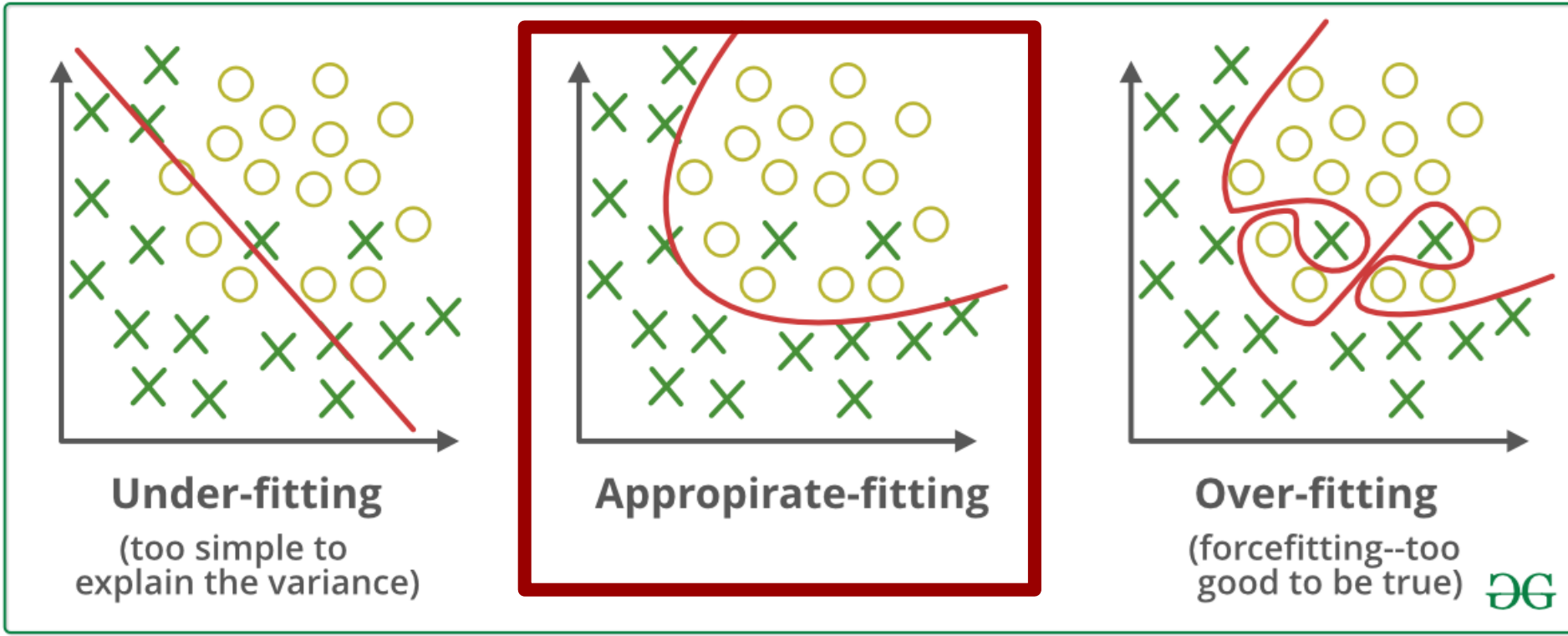
Consequences of Overfitting and Underfitting

Inaccuracy: Both lead to **unreliable predictions.**

Underfitting: High bias, the model consistently gets it wrong in a certain way.

Overfitting: High variance performance wildly depends on the specific data it has seen.

Our Goal


Under-fitting
(too simple to explain the variance)
Appropirate-fitting
Over-fitting
(forcefitting--too good to be true)

Basics - Training, Validation, and Testing Data

Training Data: We feed this data into a machine learning algorithm to develop a model.

Validation Data: This dataset tunes model parameters (hyperparameters), helping prevent overfitting.

Parameter = Learn by the model
Hyperparameters = Set by you before training

Testing Data: Provides a final, unbiased picture of how well the model performs on unseen data.

Key Evaluation Metrics (Classification)

1. **Accuracy**
2. **Precision**
3. **Recall**
4. **F1-Score**

Accuracy

It is the ratio of number of **correct predictions** to the **total number of input samples**.

$$Accuracy = \frac{Number\ of\ Correct\ predictions}{Total\ number\ of\ Predictions}$$

Note: It works well only if there are **equal** number of samples belonging to **each class**.

Accuracy- Example

Consider that there are **98%** samples of **class A** and **2%** samples of **class B** in our training set. Then our model can easily get **98%** training accuracy by simply predicting every training sample belonging to **class A**.

When the same model is tested on a test set with **60%** samples of **class A** and **40%** samples of **class B,** then the test accuracy would drop down to **60%**.

Classification Accuracy is great, but gives us the false sense of achieving high accuracy.

The real problem arises, when the cost of misclassification of the minor class samples are very high.

Possible Scenarios

True Positives (TP): Things your **model predicted positive** that were **genuinely positive**.

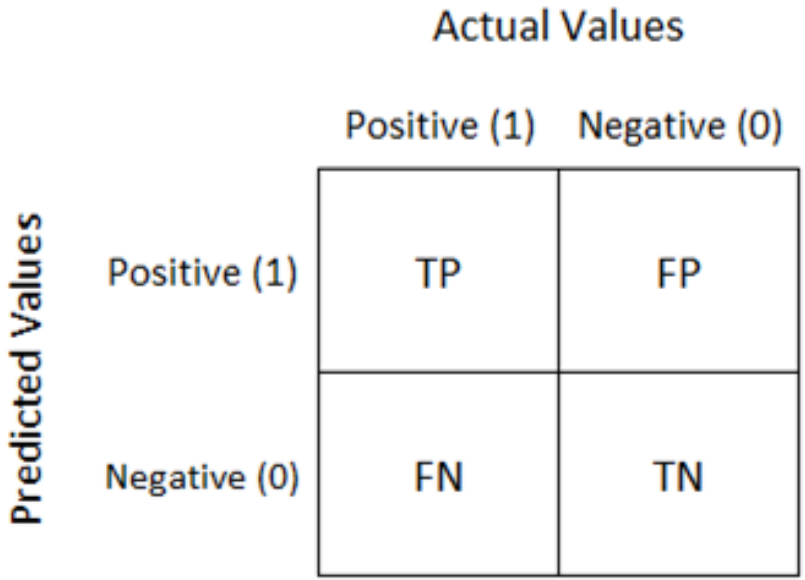
False Positives (FP): Things your **model predicted as positive**, but that were **actually negative**.

True Negatives (TN): Things **correctly** predicted as **negative**.

False Negatives (FN): Things incorrectly **predicted as negative**, when they **were positive**.

Confusion Matrix

Confusion Matrix as the name suggests gives us a matrix as output and **describes** the **complete performance** of the model.



Confusion Matrix

True Positives : The cases in which we predicted **YES** and the actual output was also **YES**.

True Negatives : The cases in which we predicted **NO** and the actual output was **NO.**

False Positives : The cases in which we predicted **YES** and the actual output was **NO**.

False Negatives : The cases in which we predicted **NO** and the actual output was **YES**.

Confusion Matrix - Accuracy

n=165	Predicted: NO	Predicted: YES
Actual: NO	50	10
Actual: YES	5	100

$$Accuracy = \frac{True\ Positive + True\ Negative}{Total\ number\ of\ Predictions}$$

$$Accuracy = \frac{50 + 100}{165} = 0.91$$

Note: Only reliable when we have balanced classes

Confusion Matrix - Precision

It is the number of **correct positive results** divided by the number of **positive results predicted** by the classifier.
precision emphasizes **minimizing false positives**.

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Postives}$$

$$Precision = \frac{50}{50 + 10} = 0.83$$

Use case example: Medical tests, where **false positives (Predicted Diseased but actually healthy)** can lead to unnecessary treatments.

Confusion Matrix - Recall

The ratio of correctly **predicted positive** instances to the **total actual positive instances**.

Recall focuses on **minimizing false negatives**.

$$Recall = \frac{True\ Positives}{True\ Positive + False\ Negative}$$

$$Recall = \frac{50}{50 + 5} = 0.91$$

Use case example: Disease detection, where **False Negative (predicted healthy but actually diseased)** can have serious consequences.

Calculate Precision - Example

Precision = TP / (TP + FP)

Precision = 35 / (35 + 10) = 35/45 = 0.78 (or 78%)

Interpretation: When the model predicts a cat has the disease, it's correct **78%** of the time. That means there's a **22%** chance of overdiagnosis.

Calculate Recall - Example

Recall = TP / (TP + FN)

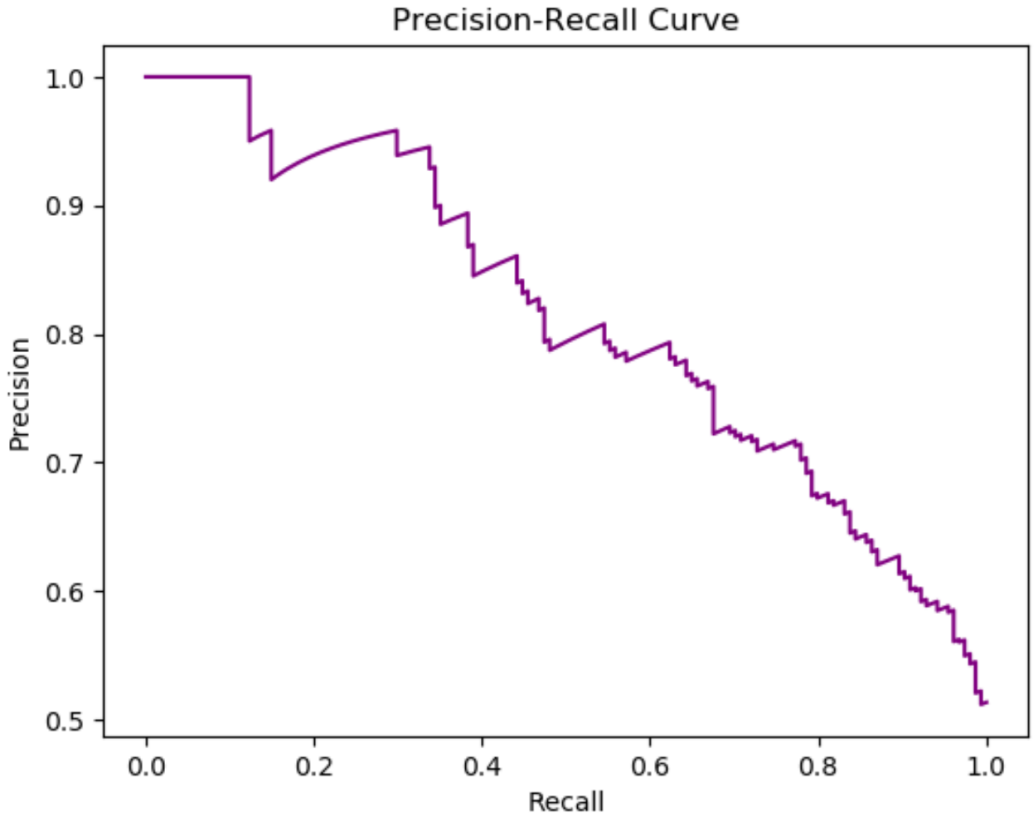
Recall = 35 / (35 + 5) = 35/40 = 0.875 (or 87.5%)

Interpretation: For cats that actually have the disease, the model correctly identifies them **87.5%** of the time. The remaining **12.5%** risk being missed by the model.

Precision Recall Curve

The **precision-recall curve** shows the **tradeoff between precision and recall** for **different threshold**. A high area under the curve represents both high recall and high precision, where high precision relates to a low false positive rate, and high recall relates to a low false negative rate.

Precision Recall Curve



Confusion Matrix - Example

Suppose you trained a model to classify cancer patients. You want evaluate the model performance. You have to answer the following questions **Accuracy?** **Precision?** and **Recall?**

What should be of more concerned Accuracy. Precision. Recall?

	Predicted Cancer = Yes	**Predicted Cancer = No**
Actual Cancer = Yes	**96**	**11**
Actual Cancer = No	**17**	**54**

Real-World Examples

Fraud detection:

High precision to avoid **false positives** (Predicted Fraud actually not-Fraud)

Medical diagnoses:

High recall to minimize **false negatives** (Predicted healthy but actually diseased).

What is the F1-Score?

- It's the **harmonic mean** of **precision** and **recall**.
- **Balances** false positives (**precision**) and false negatives (**recall**).
- Great when you need a single number to summarize model performance.
- Especially useful for **imbalanced datasets**.
- F1-score ranges between 0 and 1. The closer it is to **1**, the better the model.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

What is True Positive Rate (TPR)?

Also called "**sensitivity**" or "**recall**"

Measures the proportion of **actual positives** that the model **correctly identifies**

Formula: TPR = True Positives / (True Positives + False Negatives)

What is True Negative Rate (TNR)?

Also called "**specificity**"

Measures the proportion of **actual negatives** that the model **correctly identifies**

Formula: TNR = True Negatives / (True Negatives + False Positives)

What is False Positive Rate (FPR)?

Measures the proportion of **actual negatives** that the model **correctly identifies**

Formula: FPR= False Positive / (True Negatives + False Positives)

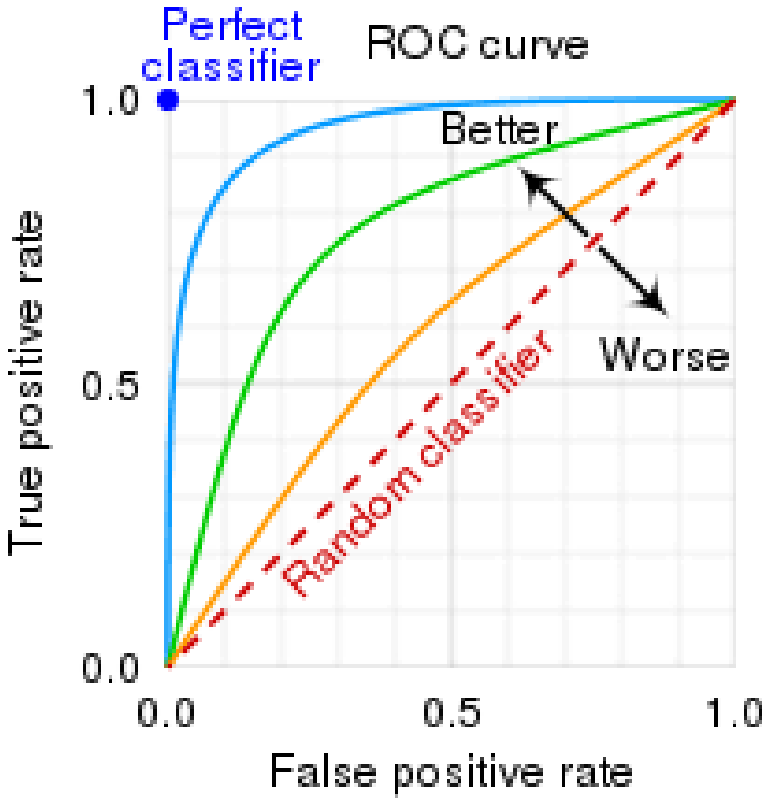
AUC-ROC Curve

ROC stands for **Receiver Operating Characteristic** curve.

It plots the **True Positive Rate (TPR)** against the **False Positive Rate (FPR)**.

Shows a model's ability to separate **positive** and **negative** examples across **different thresholds.**

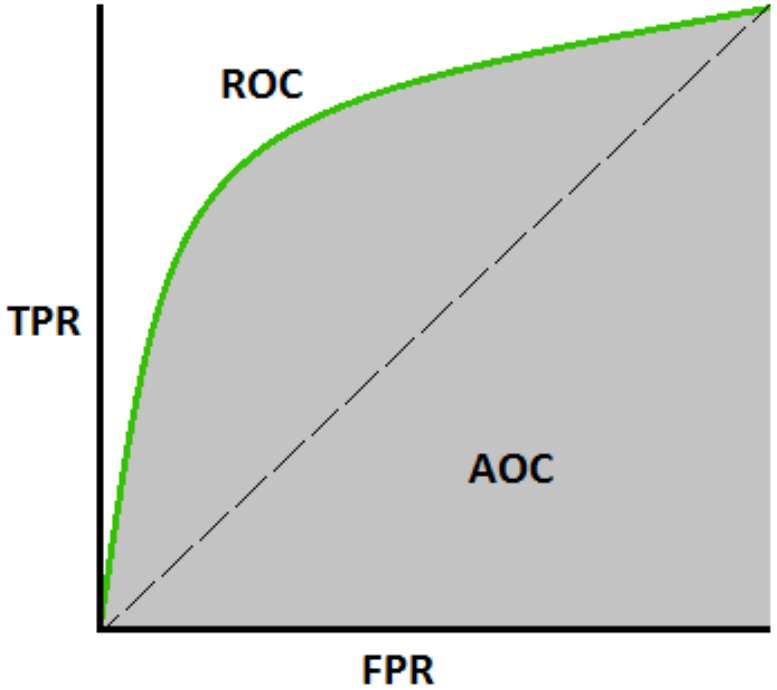
AUC-ROC Curve



What is AUC?

- AUC stands for "**Area Under the Curve**".
- It represents the model's overall ability to **distinguish classes**.
- **Larger AUC** means a **better model.**
- AUC of **1**: Perfect model.
- AUC of **0.5**: No better than random guessing.

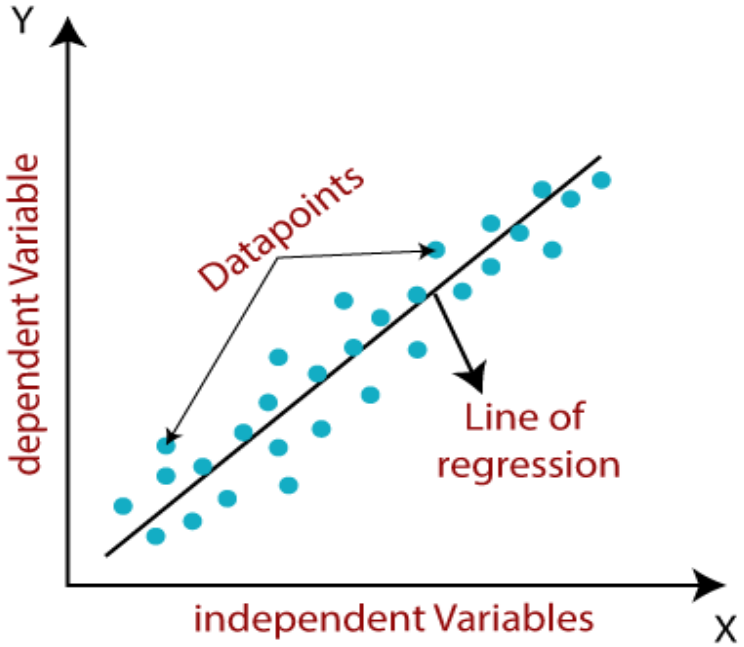
AUC-ROC Curve



Evaluation Metrics (Regression)

How does Regression work?

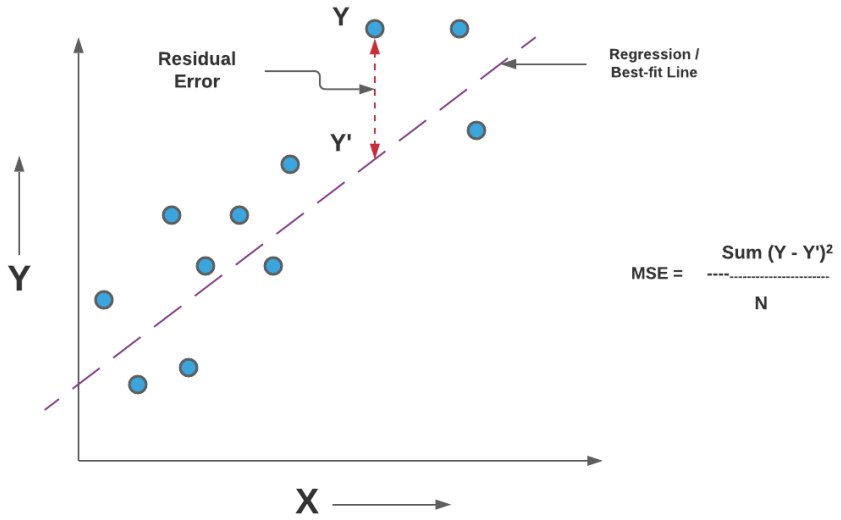
It finds the best-fitting line that represents the relationship between X and Y.



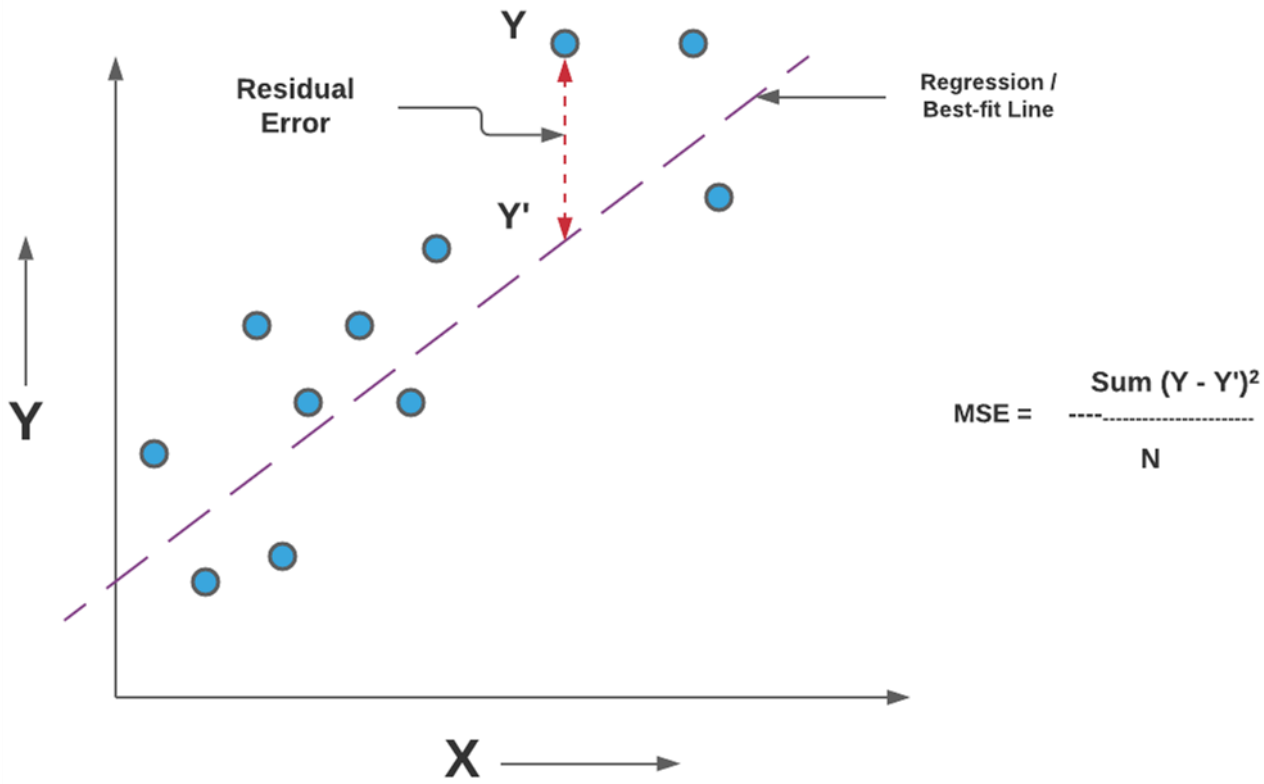
Mean Squared Error (MSE)

The **Mean Squared Error (MSE)** is a common evaluation metric used in regression tasks.

It measures the average squared difference between the actual and predicted values.



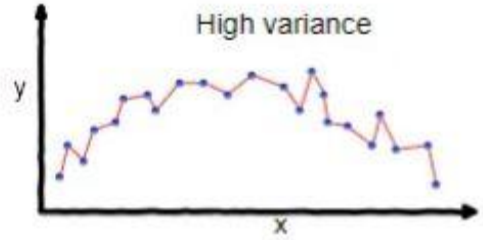
Mean Squared Error (MSE)



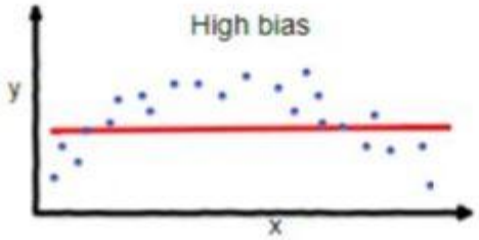
$$MSE = \frac{\text{Sum } (Y - Y')^2}{N}$$

Underfitting:

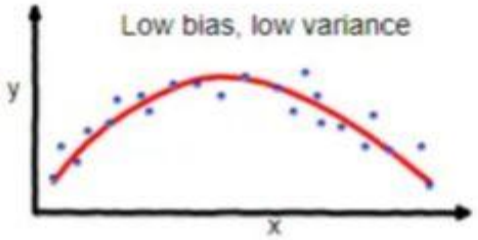
In supervised learning, **underfitting** happens when a model unable to capture the underlying pattern of the data. These models usually have high bias and low variance. It happens when we have very less amount of data to build an accurate model or when we try to build a linear model with a nonlinear data. Also, these kind of models are very simple to capture the complex patterns in data like Linear and logistic regression.



overfitting



underfitting



Good balance

Overfitting:

In supervised learning, **overfitting** happens when our model captures the noise along with the underlying pattern in data. It happens when we train our model a lot over noisy dataset. These models have low bias and high variance. These models are very complex like Decision trees which are prone to overfitting.

R-Squared (R^2)

Measures how well the model explains the variability of the target variable.

Formula:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Where:

$SS_{res} = \sum (y_i - \hat{y}_i)^2 \rightarrow$ Residual sum of squares

$SS_{tot} = \sum (y_i - \bar{y})^2 \rightarrow$ Total sum of squares

Range: 0 to 1 (can be negative if model is worse than mean)

Interpretation:

$R^2 = 1 \rightarrow$ perfect prediction
$R^2 = 0 \rightarrow$ model predicts no better than the mean
Higher $R^2 \rightarrow$ better model performance

R^2 shows how much of the data's variation the model can explain.

Adjusted R-Squared

- Measures how well the model explains the target variable while considering the number of input features.
- It adds a penalty for unnecessary features.
- Unlike R^2, it does not automatically increase when more features are added.

$$\text{Adjusted } R^2 = 1 - \left(\frac{(1 - R^2)(n - 1)}{n - p - 1}\right)$$

n= number of observations

p= number of features

Interpretation:

Higher Adjusted R^2 → better model

If Adjusted R^2 decreases after adding a feature, that feature may not be useful.

Mean Absolute Error (MAE)

Measures the **average absolute difference** between predicted values and actual values.

Gives an idea of **how far predictions are from actual values** on average.

Formula:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} | y_i - \hat{y}_i |$$

Interpretation:

Lower MAE → predictions are closer to actual values

MAE is **always non negative**

MAE measures the average absolute error between predicted and actual values.

Root Mean Square Error (RMSE)

Measures the **square root of the average squared differences** between predicted and actual values.

Gives more weight to **larger errors** than MAE.

Formula:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Interpretation:

Lower RMSE → better predictions

Sensitive to large errors due to squaring

RMSE measures the average magnitude of errors, penalizing larger mistakes more.

Cross Validation (CV)

- **Technique to evaluate model performance** on unseen data.
- Helps check if the model **generalizes well**.
- Commonly used for **hyperparameter tuning**.

How it works (k-fold CV):

- Split dataset into k equal parts (folds)
- Train model on k-1 folds, test on the remaining fold
- Repeat k times, each fold used once as test
- Take average score → overall performance

Benefits:

Reduces overfitting risk

More reliable evaluation than single train/test split

CV tests model performance on multiple folds to ensure it generalizes well.

Regularization Methods in Regression

Method	Key Idea	When & Why Use
Ridge	L2 penalty → shrinks coefficients	Use when **many small/medium correlated features**; reduces overfitting but keeps all features
Lasso	L1 penalty → can shrink coefficients to zero	Use when **only some features matter**; performs **feature selection**
Elastic Net	Combination of L1 + L2	Use when **features are correlated** and you want **both shrinkage + selection**